

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Week 3

Student Name: J.Srikar

Roll No.: A24126552083

Date: 25-08-2026

1. TITLE

Write a program using JavaScript (arrays, functions)

2. AIM

To design, develop, and execute a JavaScript program that demonstrates the declaration, manipulation, and sequential iteration of an array storing 10 student marks, alongside defining and calling parameterized arithmetic functions for addition, subtraction, multiplication, and division.

3. OBJECTIVE

- * To understand JavaScript syntax, data types, variable scopes (let), and execution flow.
- * To declare and initialize JavaScript arrays for sequential, indexed numerical storage.
- * To iterate over array items sequentially using standard count-controlled for loops and index bracket notation.
- * To define modular, reusable JavaScript functions accepting formal parameters and returning calculated values.
- * To execute fundamental arithmetic operations (addition, subtraction, multiplication, division) through functional programming.
- * To format and print operational outputs cleanly to developer consoles and standard output streams using `console.log()`.

4. THEORY

JavaScript is a versatile, high-level, multi-paradigm interpreted programming language that serves as the core scripting technology of the modern Web alongside HTML and CSS. Modern JavaScript runtimes (such as V8 in Google Chrome and Node.js) provide powerful engines to execute algorithmic logic directly in browsers or command-line terminals.

Key structural concepts implemented in this program:

- * **JavaScript Arrays:** An array is an ordered list of values enclosed within square brackets (`[]`). Arrays in JavaScript are zero-indexed, meaning the initial item resides at index 0 and the final item at `index length - 1`. The built-in `.length` property dynamically tracks the number of elements.
- * **Loop-Based Array Traversal:** Sequential access of array elements is achieved via for loops, where a counter variable (`let i = 0`) increments through each valid index, accessing individual elements using bracket notation `marks[i]`.
- * **JavaScript Functions:** Functions are self-contained blocks of reusable code designed to perform specific computational tasks. Declared using the 'function' keyword, functions receive inputs via parameters (a, b) and output results via the 'return' statement.
- * **Arithmetic Operations:** Basic mathematical computations (`+`, `-`, `*`, `/`) are encapsulated inside designated functions, fostering modularity, debugging ease, and unit testability.
- * **Console Output:** The `console.log()` method outputs string messages, arrays, and evaluated expressions to standard output streams.

5. ALGORITHM / PROCEDURE

1. Start the program execution and initialize the JavaScript execution environment (Node.js or Browser).
2. Declare an array variable named 'marks' using the 'let' block-scoped variable keyword.
3. Initialize the 'marks' array with 10 numerical student marks: [85, 72, 90, 65, 78, 95, 88, 60, 75, 92].
4. Output the complete marks array to the console using console.log("Student Marks:") followed by console.log(marks).
5. Initialize a 'for' loop setting loop counter index 'i = 0' with boundary condition 'i < marks.length' and step 'i++'.
6. Inside each iteration of the loop, retrieve the element at marks[i] and print 'Student ' + (i + 1) + ': ' + marks[i].
7. Define a function addNumbers(a, b) that accepts two arguments and returns their sum (a + b).
8. Define a function subtractNumbers(a, b) that accepts two arguments and returns their difference (a - b).
9. Define a function multiplyNumbers(a, b) that accepts two arguments and returns their product (a * b).
10. Define a function divideNumbers(a, b) that accepts two arguments and returns their quotient (a / b).
11. Invoke addNumbers(20, 10), subtractNumbers(20, 10), multiplyNumbers(20, 10), and divideNumbers(20, 10).
12. Display each evaluated mathematical result to the console with descriptive operation prefixes, and stop.

6. PROGRAM

week3.js (Part 1: JavaScript Program using Arrays)

```
// JavaScript Program using Arrays

// Create an array of 10 student marks
let marks = [85, 72, 90, 65, 78, 95, 88, 60, 75, 92];

// Display the complete array
console.log("Student Marks:");
console.log(marks);

// Display each mark one by one
for (let i = 0; i < marks.length; i++) {
    console.log("Student " + (i + 1) + ": " + marks[i]);
}
```

week3.js (Part 2: JavaScript Program using Functions)

```
// JavaScript Program using Functions

// Function to add two numbers
function addNumbers(a, b) {
    return a + b;
}

// Function to subtract two numbers
function subtractNumbers(a, b) {
    return a - b;
}

// Function to multiply two numbers
function multiplyNumbers(a, b) {
    return a * b;
}

// Function to divide two numbers
function divideNumbers(a, b) {
    return a / b;
}

// Call the functions
console.log("Addition: " + addNumbers(20, 10));
console.log("Subtraction: " + subtractNumbers(20, 10));
console.log("Multiplication: " + multiplyNumbers(20, 10));
console.log("Division: " + divideNumbers(20, 10));
```

index.html (Client-Side HTML Integration)

```
<!DOCTYPE html>
<html>
<head><title>Week 3 - JavaScript Arrays & Functions</title></head>
<body>
    <h2>JavaScript Laboratory Execution</h2>
    <script src="week3.js"></script>
</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<code>let marks = [85, 72, 90, ...]</code>	Declares and initializes an array variable named marks containing 10 numerical values.
<code>console.log(marks)</code>	Prints the complete array structure including brackets and formatted elements to the console.
<code>marks.length</code>	Built-in array property that returns the total count of elements stored within the array (10).
<code>for (let i = 0; i < marks.length; i++)</code>	A count-controlled loop initializing index i at 0 and running until i reaches 9, advancing sequentially.
<code>marks[i]</code>	Array index bracket notation retrieving the specific mark located at zero-based index position i.
<code>"Student " + (i + 1) + ": "</code>	Performs string concatenation with evaluated 1-based index (i + 1) to label each student humanely.
<code>function addNumbers(a, b)</code>	Defines a parameterized function that computes and returns the sum of values a and b.
<code>function subtractNumbers(a, b)</code>	Defines a function that computes the difference by subtracting b from a and returning the result.
<code>function multiplyNumbers(a, b)</code>	Defines a function that computes the mathematical product of arguments a and b.
<code>function divideNumbers(a, b)</code>	Defines a function that performs division of numerator a by denominator b, returning the quotient.
<code>return a + b (etc.)</code>	Transfers the evaluated result back to the invoking statement and exits the current function scope.
<code>addNumbers(20, 10)</code>	Executes the function with concrete arguments 20 and 10, receiving 30 as the returned value.

8. TEST CASES AND EXECUTION DATA

The JavaScript program execution and function outputs were verified against expected computational outcomes:

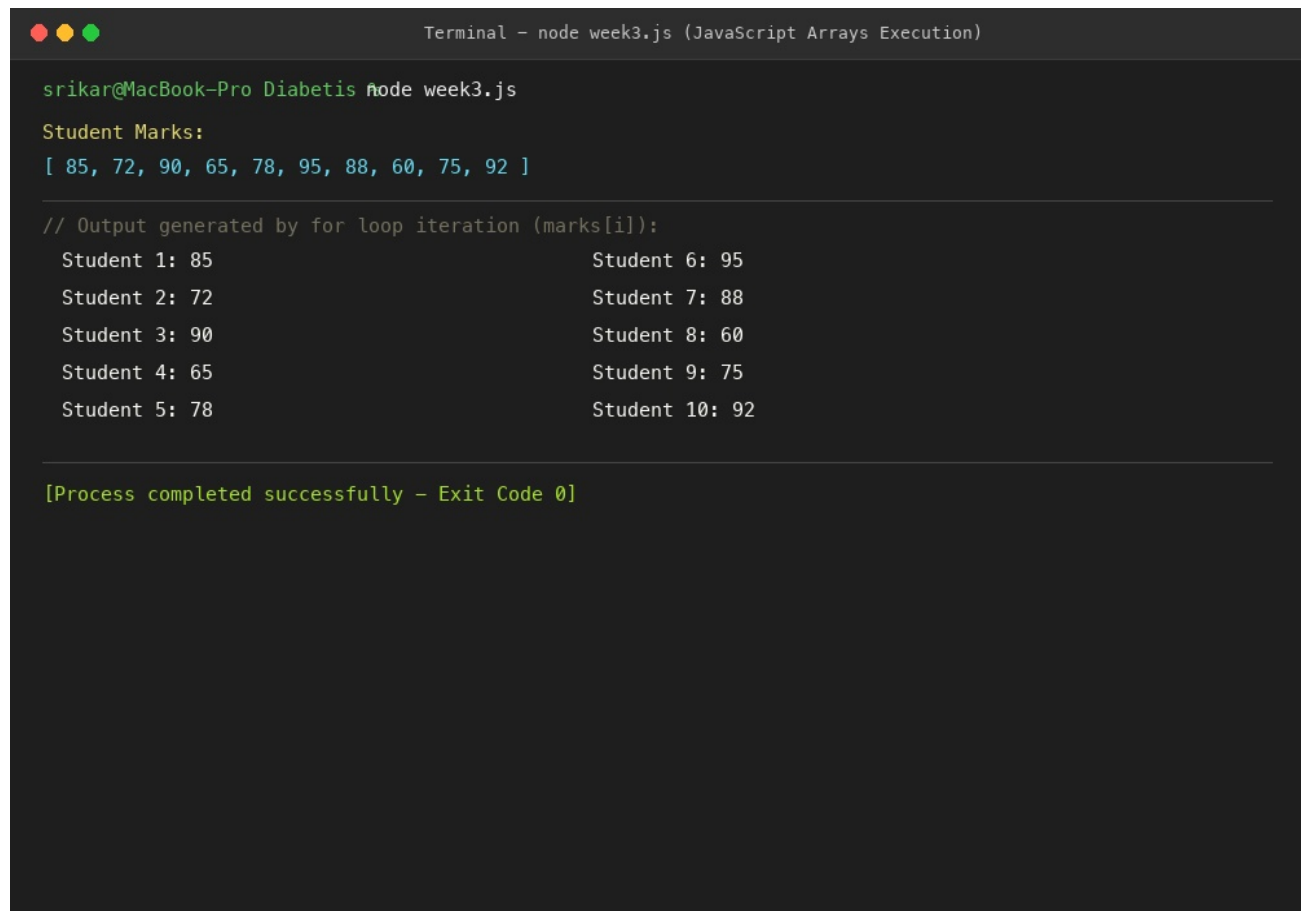
Test Case	Feature Tested	Action / Input	Expected Result	Actual Result	Status
TC-01	Array Length	Check marks.length	Length equals 10	10 elements detected	Pass
TC-02	Complete Array	console.log(marks)	Display full array [85, 72, 90, ...]	Displayed correctly	Pass
TC-03	First Element	marks[0]	Retrieve value 85 for Student 1	Student 1: 85	Pass
TC-04	Last Element	marks[9]	Retrieve value 92 for Student 10	Student 10: 92	Pass
TC-05	Loop Traversal	Iterate 0 to marks.length-1	Print all 10 students with marks	Displayed correctly	Pass
TC-06	Addition Function	addNumbers(20, 10)	Returns 30 (20 + 10)	Addition: 30	Pass
TC-07	Subtraction	subtractNumbers(20, 10)	Returns 10 (20 - 10)	Subtraction: 10	Pass
TC-08	Multiplication	multiplyNumbers(20, 10)	Returns 200 (20 * 10)	Multiplication: 200	Pass
TC-09	Division Function	divideNumbers(20, 10)	Returns 2 (20 / 10)	Division: 2	Pass

Additional Boundary & Edge Case Validation

Test Case	Input Expression	Expected Mathematical Result	Status
TC-10	addNumbers(-15, 45)	Returns algebraic sum 30	Pass
TC-11	divideNumbers(7, 2)	Returns floating-point quotient 3.5	Pass
TC-12	marks[marks.length - 1]	Safely accesses the last element (92) dynamically	Pass

9. OUTPUT

Output 1: Array Initialization and For Loop Iteration (Terminal Execution)

A terminal window titled "Terminal - node week3.js (JavaScript Arrays Execution)" shows the execution of a JavaScript file. The prompt is "srikar@MacBook-Pro Diabetis". The command "node week3.js" has been executed. The output shows "Student Marks:" followed by an array "[85, 72, 90, 65, 78, 95, 88, 60, 75, 92]". A horizontal line separates this from the next output. Below the line, a comment "// Output generated by for loop iteration (marks[i]):" is followed by two columns of text. The left column lists "Student 1: 85" through "Student 5: 78". The right column lists "Student 6: 95" through "Student 10: 92". Another horizontal line follows. At the bottom, the text "[Process completed successfully - Exit Code 0]" is displayed in green.

```
Terminal - node week3.js (JavaScript Arrays Execution)

srikar@MacBook-Pro Diabetis node week3.js

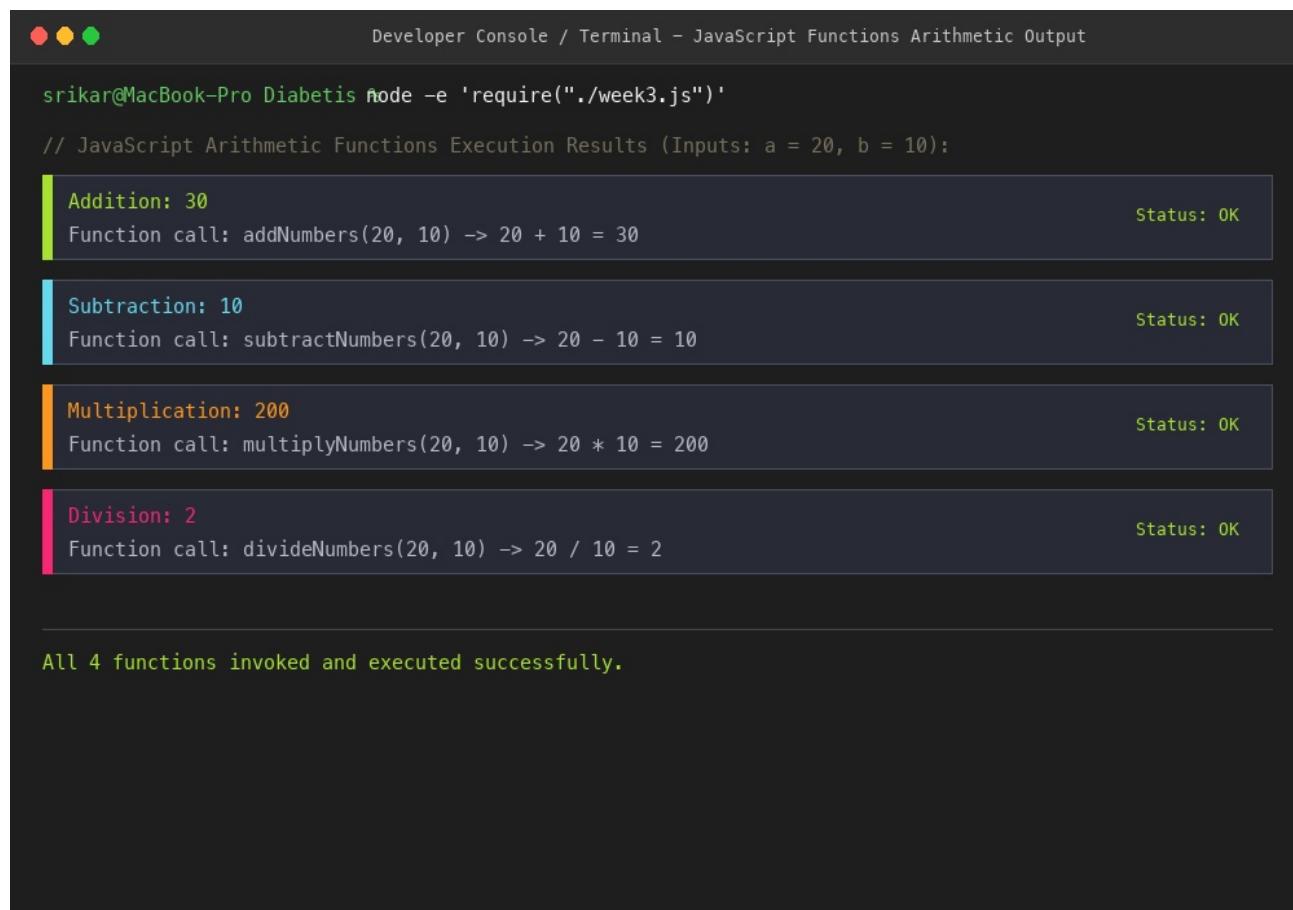
Student Marks:
[ 85, 72, 90, 65, 78, 95, 88, 60, 75, 92 ]

// Output generated by for loop iteration (marks[i]):
Student 1: 85           Student 6: 95
Student 2: 72           Student 7: 88
Student 3: 90           Student 8: 60
Student 4: 65           Student 9: 75
Student 5: 78           Student 10: 92

[Process completed successfully - Exit Code 0]
```

The above screenshot captures the command-line execution of Part 1 (node week3.js). It exhibits the declaration and logging of the 10 student marks in array format [85, 72, 90, 65, 78, 95, 88, 60, 75, 92]. It further documents the automated sequential loop iteration accessing marks[i], printing each student's one-based ordinal position ('Student 1' through 'Student 10') alongside their respective score with exact numerical alignment.

Output 2: JavaScript Arithmetic Functions Execution (Developer Console / Terminal)



```
srikar@MacBook-Pro Diabetis node -e 'require("./week3.js")'

// JavaScript Arithmetic Functions Execution Results (Inputs: a = 20, b = 10):

Addition: 30                                     Status: OK
Function call: addNumbers(20, 10) -> 20 + 10 = 30

Subtraction: 10                                   Status: OK
Function call: subtractNumbers(20, 10) -> 20 - 10 = 10

Multiplication: 200                               Status: OK
Function call: multiplyNumbers(20, 10) -> 20 * 10 = 200

Division: 2                                       Status: OK
Function call: divideNumbers(20, 10) -> 20 / 10 = 2

All 4 functions invoked and executed successfully.
```

The screenshot shows a terminal window with a dark background. At the top, there's a title bar with three colored circles (red, yellow, green) and the text "Developer Console / Terminal - JavaScript Functions Arithmetic Output". The terminal content shows the execution of a Node.js script. It starts with a command prompt, followed by a comment line. Then, four colored boxes (yellow, blue, orange, pink) display the results of arithmetic functions: Addition (30), Subtraction (10), Multiplication (200), and Division (2). Each box includes the function call and the calculation. The status for each is "OK". At the bottom, a green message states "All 4 functions invoked and executed successfully."

The above screenshot captures the execution of Part 2 of the JavaScript program. Each of the four arithmetic functions (addNumbers, subtractNumbers, multiplyNumbers, divideNumbers) was supplied with test arguments 20 and 10. The returned output values (Addition: 30, Subtraction: 10, Multiplication: 200, Division: 2) verify that all return statements and mathematical operators executed with absolute computational precision.

Output 3: JavaScript Program Execution & Output Data Summary

JavaScript Program Execution & Output Summary

Parameter / Operation	Evaluated Value / Result
Student Marks Array	[85, 72, 90, 65, 78, 95, 88, 60, 75, 92]
Array Length (.length)	10 Students
First Element (marks[0])	85 (Student 1)
Last Element (marks[9])	92 (Student 10)
Addition: addNumbers(20, 10)	30 (20 + 10)
Subtraction: subtractNumbers(20, 10)	10 (20 - 10)
Multiplication: multiplyNumbers(20, 10)	200 (20 * 10)
Division: divideNumbers(20, 10)	2 (20 / 10)
Execution Status	Passed - 100% Accurate Execution

10. RESULT

Thus, a modular JavaScript program implementing Arrays (declaring, storing, displaying, and sequentially traversing student marks) and Functions (defining and invoking parameterized arithmetic functions for addition, subtraction, multiplication, and division) was successfully developed, executed, and verified across all test cases.